

Отчёт по аудиту безопасности веб-приложения

Анкета разработчика с авторизацией и админ-панелью
База данных: u82296

1. XSS (Cross-Site Scripting)

Уязвимость позволяет злоумышленнику внедрить JavaScript-код через поля формы. При выводе данных в HTML код выполняется в браузере жертвы.

Методы защиты:

Все пользовательские данные при выводе обрабатываются функцией `htmlspecialchars()`, которая превращает опасные символы `<`, `>`, `"`, `'` в безопасные HTML-сущности. На входе данные проходят `trim()` и валидацию регулярными выражениями.

Пример — экранирование при выводе (form.php):

```
value="<?= htmlspecialchars($values['fio'] ?? '') ?>"
```

Пример — валидация на входе (index.php):

```
if (!preg_match('/^[a-zA-Za-яА-ЯёЁ\s\-\_]+$/', $fio))  
    return 'ФИО содержит недопустимые символы';
```

Пример — экранирование в админке (admin.php):

```
<td><?= htmlspecialchars($app['fio']) ?></td>
```

2. Information Disclosure (Раскрытие информации)

При ошибках подключения к БД пользователю показывались технические детали: имя базы, хост, стек вызовов — это помогает злоумышленнику спланировать атаку.

Методы защиты:

Все блоки `try-catch` заменены на нейтральные сообщения без технических деталей. Вызов `$e->getMessage()` удалён из пользовательского вывода.

Было (index.php, login.php, admin.php):

```
die("Ошибка подключения к базе данных: " . $e->getMessage());
```

Стало:

```
die("Ошибка сервера. Попробуйте позже.");
```

Дополнительно: пароли в базе хранятся как `bcrypt`-хеши через `password_hash()`. Исходный пароль невосстановим из хеша.

3. SQL Injection

Злоумышленник через поля формы может вставить SQL-код, изменяющий логику запроса — например, получить доступ без пароля или удалить все данные.

Методы защиты:

Все SQL-запросы выполняются через Prepared Statements (prepare + execute). Данные передаются отдельно от SQL-кода, что исключает внедрение.

Пример — вход пользователя (login.php):

```
$stmt = $pdo->prepare("SELECT id, login, password_hash
FROM task5_applications WHERE login = ?");
$stmt->execute([$login]);
```

Пример — удаление в админке (admin.php):

```
$pdo->prepare("DELETE FROM task5_applications WHERE id = ?")
->execute([$id]);
```

Пример — проверка по белому списку (index.php):

```
if (!in_array($lang, $allowedLanguages))
    return 'Выбран недопустимый язык';
```

4. CSRF (Cross-Site Request Forgery)

Злоумышленник может создать страницу с формой, автоматически отправляющей POST-запрос на наш сайт от имени авторизованного пользователя.

Методы защиты:

Реализована полноценная CSRF-защита: генерация токена при старте сессии, скрытое поле в каждой форме, проверка через hash_equals() при каждом POST.

Было:

CSRF-защита отсутствовала во всех формах.

Стало — генерация токена (index.php, login.php, admin.php):

```
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
```

Стало — скрытое поле в форме (form.php, login.php, admin.php):

```
<input type="hidden" name="csrf_token"
value="<?= htmlspecialchars($_SESSION['csrf_token'] ?? '') ?>">
```

Стало — проверка при POST (index.php):

```
if (!isset($_POST['csrf_token'])
|| !hash_equals($_SESSION['csrf_token'] ?? '', $_POST['csrf_token'])) {
    http_response_code(403);
    die('Ошибка безопасности: недействительный CSRF-токен.');
```

Стало — удаление в админке через POST вместо GET (admin.php):

Было: `<a href="?delete=id">` — удаление через GET-ссылку.

Стало: `<form method="POST">` с `csrf_token` и `delete_id`.

5. Include (Подключение файлов)

Если в `include()` используется пользовательский ввод, злоумышленник может подключить произвольный файл.

Методы защиты:

В приложении `include()` используется только с жёстко прописанным именем файла `include('form.php')`. Путь не зависит от пользовательского ввода. Подключается только локальный доверенный файл-шаблон.

Пример из index.php:

```
include('form.php'); // Жёстко прописанный путь
```

6. Upload (Загрузка файлов)

Если приложение позволяет загружать файлы, злоумышленник может загрузить PHP-скрипт и выполнить его на сервере.

Методы защиты:

В приложении загрузка файлов не реализована. Ни одна форма не содержит `<input type="file">`. Атрибут `enctype="multipart/form-data"` не используется. Массив `$_FILES` не обрабатывается. Уязвимость отсутствует конструктивно.